

Can AI and Multi-Agent Systems Develop and Debug Robotics Software?

1. Title and Abstract

Title: Strong at the Code, Weak at the Body: A First-Iteration Study of AI and Multi-Agent Systems Developing and Debugging Robotics Software.

Abstract (TL;DR): This document tests a single working thesis: AI and multi-agent systems are strong at the software-engineering side of robotics (writing, repairing, and debugging ROS/ROS2 code through closed-loop iteration) but weak as the runtime embodied brain (low-level sensorimotor control, spatial reasoning, multi-robot coordination, and realtime error recovery). We approach the question from two evidence sources held to a single standard: every factual claim must trace either to a literature source (whose URL was adversarially fetched and confirmed, or to a citation in our own conversation corpus (file, date, turn) backed by a raw transcript. On the literature side we synthesized 78 verified findings spanning general software repair, robotics-specific code and debugging, embodied-control benchmarks, named agentic tools in robotics, and benchmark-reliability caveats. On the primary-evidence side we analyze 30 of our own Claude Code sessions (2,153 user turns, 4,192 assistant turns, 1,697 tool calls) from a real TurtleBot 4 capstone project. The headline finding is corroboration: across the 28 non-empty sessions, the agent reached or partially reached its goal in the overwhelming majority, and every single session classified as touching robot software engineering supported the thesis, while every place the work crossed into the physical runtime (verifying a GUI rendered, confirming the robot actually moved, docking, surviving a camera-rail power brownout) is exactly where the agent could not close the loop on its own. Our logs agree with the literature. The split is not a deficiency of one model on one day; it is a structural boundary between reasoning about code that controls a robot and being the controller.

2. Goal of the Research

The research question is: **Can AI and multi-agent systems develop and debug robotics software?**

The working thesis being tested is more precise than a yes or no. It claims an asymmetry. AI and agent systems are expected to be strong at the software-engineering side of robotics, meaning authoring ROS and ROS2 nodes, configuration, and launch files, and repairing and debugging them through closed-loop write-run-read-error-fix iteration. They are expected to be weak as the runtime embodied brain, meaning low-level sensorimotor control, spatial reasoning, multi-robot or multi-agent coordination at runtime, and recovery from physical execution errors.

The goal of iteration 1 is to test whether that asymmetry holds when we put it against two independent kinds of evidence: the published literature, and our own working sessions on a real robot. If both agree, the thesis is corroborated and the boundary becomes a design constraint we can build around. If they diverge, the divergence itself is the finding.

3. Methodology

3.1 Two evidence streams, one standard

Every claim in this document is held to one rule: it must be traceable. A claim about the literature must cite a source whose URL appears in [Research/Literature/Findings.md](#). A claim about our own work must cite a session as ([file](#), [date](#), [turn](#)) and refer to the raw transcript under [Claude-conversations/sessions/](#). Corpus statistics use only the measured numbers in [Research/conversation-analysis/INDEX.md](#) and [stats.json](#). Where we were tempted to add outside knowledge that was not in these inputs, we omitted it.

3.2 The conversation corpus: collection, redaction, deterministic digest

The primary evidence is our own usage. The sessions were collected by a configurable, tested collector built and exercised inside the project itself. Two sessions in the corpus document that very thing being built: a reusable cross-laptop tool to git, Claude Code transcripts, strip internal metadata from the JSONL, redact secrets, store the result in collect, and push it (file: 2026-06-22_610e67b0.jsonl, date: 2026-06-22, turn: 1), and a later run of the resulting [collect-claude-conversations](#) skill across all conversation folders on the device (file: 2026-06-22_6014c160.jsonl, date: 2026-06-22, turn: 1). Redaction is not an afterthought. The collector runs a credential scan; in one build the proactive secret scan found a live AWS access key ID and a real password and the agent refused to push until they were removed, with an independent grep confirming zero instances of the real secrets remained (file: 2026-06-22_610e67b0.jsonl, date: 2026-06-22, turn: 81, 148).

From the redacted JSONL we produced a deterministic digest per session and a [stats.json](#) of measured counts (raw bytes, user turns, assistant turns, tool calls, images, and a top-tokens histogram). These counts are mechanical, not judgments. The corpus totals are: 30 sessions analyzed, 2,153 user turns, 4,192 assistant turns, 1,697 tool calls, with 178 tasks recorded done and 46 recorded failed or abandoned (INDEX.md, corpus totals).

3.3 Agent analysis of each conversation

Each digest was analyzed and the analysis recorded in [_analyses.json](#) with a fixed schema: motive, outcome (reached, partial, not-reached), thesis category (software-engineering, mixed, non-technical, embodied-hardware), automation level, the measured turn and tool citations, a thesis-alignment paragraph, and a list of [key_evidence](#) items each carrying a ([file](#), [date](#), [turn](#)) citation, a thesis-back to the raw transcript. This lets every interpretive statement in Section 5 be checked against a specific point in a specific session.

3.4 The literature: research then adversarial URL verification

The literature began from a preliminary review ([compass_artifact...md](#)) and was then rebuilt under verification. Finder agents searched for evidence per cluster. Every candidate finding was sent to two independent skeptic agents that fetched the cited URL and judged whether the page actually supported the claim. A finding is kept only if at least one skeptic confirmed support and none refuted it (verification-log.md, Method). Of 91 candidates, 78 were kept and 13 dropped, an 86 percent keep rate (verification-log.md). This is an adversarial standard: a kept finding has a source URL that was actually retrieved and read.

The dropped findings are instructive about why the standard matters. Several attractive numbers failed verification: a claimed "0.40 to 0.80, exceeds 0.60 human baseline" RoboCoach result was refuted because the source reports a 26.5 percent margin and no such figures (verification-log.md, RoboCoach drop); a precise EmbodiedBench "28.9 percent on low-level manipulation" framing was downgraded to cannot verify because the abstract attributes 28.9 percent to GPT-4o's overall average, not to low-level manipulation specifically (verification-log.md, EmbodiedBench drop); and a "Silco-Bench" claim was caught when deterministic PDF extraction showed earlier fetch summaries had hallucinated the supporting text (verification-log.md, Silco-Bench drop). Because of this, the numbers we cite below are the ones that survived.

3.5 Acknowledged gaps from the completeness critic

The same process flagged where the thesis is least supported by direct evidence. Multi-robot coordination as a runtime phenomenon is thin: PARTNR touches human-robot collaboration and ROSClaw did not test multi-robot coordination. The perception-versus-reasoning confound is real: several strong embodied results assume perception is solved via privileged state. Error recovery as a distinct runtime weakness is scattered across findings rather than isolated. Sim-to-real validation depth is uneven. These gaps are carried forward honestly into Limitations (Section 7).

4. Literature Review

4.1 General software engineering: code generation, repair, and debugging

The trajectory on general software repair is steep. When SWE-bench launched (ICLR 2024) the best frontier model (Claude 2) resolved only 1.96 percent of 2,294 real GitHub issue task instances, showing models initially could solve almost none of these realistic repository-level tasks ([https://openreview.net/pdf?id=VTF8yNQM66](#)). Multi-agent debuggers then posted strong gains. FixAgent/UniDebugger, a hierarchical multi-agent framework, set a new state of the art on Defects4J by correctly fixing 197 bugs (286 plausible fixes), beating prior SoTA ChatRepair by about 25.48 percent and fixing all bugs in QuixBugs across Java and Python ([https://arxiv.org/abs/2405.17153](#)), and fixes 1.25x to 2.56x more bugs than prior methods on the repository-level Defects4J benchmark without prior fault localization ([https://aclanthology.org/2025.emnlp-main.921/](#)). RepairAgent's closed-loop write-run-validate iteration is cheap, averaging 270,000 tokens per bug, about 14 cents at GPT-3.5 pricing ([https://arxiv.org/abs/2403.17134](#)). RGD, a three-agent (Guide, Debug, Feedback) debugger, reached state of the art with a 9.8 percent improvement on HumanEval and 16.2 percent on MBPP ([https://arxiv.org/abs/2410.01242](#)), and ablations show the decomposition itself matters: removing the Guide Agent drops HumanEval by 4.4 percent and MBPP by 6.4 percent ([https://arxiv.org/html/2410.01242v2](#)). CodeSim reached pass@1 of 95.1 percent on HumanEval and 90.7 percent on MBPP ([https://arxiv.org/abs/2502.05664](#)).

The mechanism behind these gains is closed-loop iteration, and the residual weakness is semantic, not syntactic. Failure analysis generated patches syntactics logic errors (47 percent) and incomplete fixes (35 percent) to the bulk of failures, while surface syntactic errors account for only about 14 percent ([https://arxiv.org/pdf/2603.00520](#)). Multi-agent frameworks also carry costs: ChatDev reports high executability (0.88) but only moderate completeness (0.56), and its authors warn the systems suit prototypes rather than complex real-world applications ([https://arxiv.org/html/2307.07924v5](#)), while MetaGPT reaches 85.9 percent Pass@1 on HumanEval but consumes 31,255 tokens per task versus ChatDev's 19,292 ([https://arxiv.org/html/2308.00352v6](#)).

4.2 Robotics-specific code generation and debugging

The strength carries into robotics code, with the same closed-loop signature. OperateLLM integrates ROS2 tools into an LLM via ReAct and generates fundamental ROS2 components such as Nodes and Publishers ([https://eeexplore.ieee.org/document/10730448/](#)), and LLM-based agents can read and reason over real ROS2 codebases, recovering hierarchical architecture models directly from source code and launch files across three repositories including an industrial-scale subset ([https://arxiv.org/abs/2602.18644](#)). A closed-loop generate-simulate-evaluate framework for drone/UAV code raised task success from 33.3 to 55 percent (direct generation) up to 85 to 93.3 percent with iterative error-fix correction in AirSim and PX4-Gazebo ([https://arxiv.org/html/2512.02002v1](#)), direct evidence that write-run-read-error-fix loops sharply improve real robot control code. RoboRepair traces execution of an LLM-generated program up to failure and runs an LLM-produced recovery program over eleven error-condition tasks ([https://arxiv.org/html/2410.18893](#)). The grey-literature robotics-agent-skills package reports skills-enabled generation producing 601 lines of tests (versus 0), upgrading plain nodes to lifecycle nodes, and expanding configuration from 4 to 13 parameters ([https://github.com/arpiqtq1304/robotics-agent-skills](#)).

The most thesis-relevant robotics result is a detection-versus-root-cause gap. ROS Help Desk achieved 100 percent accuracy at detecting which injected error type occurred but only 18 percent at identifying the specific injected fault (root cause) ([https://arxiv.org/abs/2507.07846](#)), and it averaged 68 percent across eight debugging criteria while its ROSA baseline reached only 29 percent detection ([https://arxiv.org/html/2507.07846v1](#)). This is the software-engineering strength and its ceiling in one experiment: near-perfect surface detection, weak deep diagnosis. RoboEval makes the same point on the generation side: even GPT-4 reached pass@1 of at least 0.75 on up to 67.5 percent of prompts, with hallucinated invalid locations or objects dominating execution errors (only 80.6 percent for StarCoder) ([https://arxiv.org/abs/2311.11183](#)). Closed-loop policy development can be very strong: RoboCoach-Bench (32 tasks, four simulators) shows the agentic loop matching and in aggregate exceeding expert-authored references under binary task-completion criteria ([https://arxiv.org/html/2601.21570](#)), with the gains coming from the closed-loop machinery (execution feedback, rollout-video diagnostics, memory and search), surpassing human-engineered baselines by 26.5 percent ([https://arxiv.org/abs/2601.21570](#)), and the optimized policies transferring to hardware, improving real-robot success from 0.51 to 0.68 averaged over four transfer tasks ([https://arxiv.org/html/2601.21570](#)). FAEA applies an unmodified Claude Agent SDK directly to embodied manipulation using the same iterative debugging loop, reaching 84.9 percent (LIBERO), 85.7 percent (ManiSkill3), and 96 percent (MetaWorld) with privileged state access and zero demonstrations ([https://arxiv.org/abs/2601.20334](#)).

4.3 Embodied control: where LLMs and VLMs remain weak

When the model is asked to be the runtime brain rather than to write the code, the picture inverts. On EmbodiedBench, GPT-4o scores 52 percent (ALFRED) and 36 percent (Habitat) on spatial-awareness subsets at the high level but only 25.0 percent on the EB-Manipulation spatial-awareness subset and 19.4 percent on its visual-apprecance subset ([https://arxiv.org/html/2502.09560v1](#)). On Butter-Bench (real robot, 95% as high-level controller), the best LLM (Gemini 2.5 Pro) scores 40 percent versus a mean human score of 9.9 percent, with Claude Opus 4.1 at 37 percent and Llama 4 Maverick at 7 percent ([https://arxiv.org/html/2510.21860v1](#)). On PARTNR (100,000 multi-agent human-robot collaboration tasks), humans solve 93 percent but SoTA LLMs complete only 30 percent under non-privileged conditions, with documented failures in coordination, task tracking, and error recovery ([https://arxiv.org/abs/2411.00081](#)). On RoboBench, the strongest model still lags humans and all models degrade by about 30 percent on implicit instructions ([https://arxiv.org/abs/2510.17801](#)). On VLBench, SoTA VLMs perform near zero on composite tasks (Octo 0.00 percent, OpenVLA 2.66 percent, RLT-B1 3.34 percent) ([https://arxiv.org/html/2412.18194v1](#)). On BLINK, humans average 95.70 percent on core spatial perception while GPT-4V reaches only 51.26 percent and Gemini 45.72 percent ([https://arxiv.org/abs/2404.12390](#)).

Three runtime failure modes recur. **Coordination:** on PARTNR, decentralized two-agent LLM coordination is worse than a single agent, taking about 31 percent more simulation steps (3295 versus 2519) and a 300 percent increase in extraneous effort, a coordination "burden" ([https://arxiv.org/abs/2411.00081](#)); a re-evaluation found one robot working while the other sat idle even when high-level completion looked decent ([https://arxiv.org/abs/2507.06157](#)), and DPBench shows agents handle sequential but not simultaneous coordination, with N=5 deadlock from 25.0 percent (GPT-5.2) to 90.0 percent (Gemini 2.5 Flash), governed by the interaction protocol rather than model capability ([https://arxiv.org/abs/2602.13255](#)). **Error recovery:** ROSClaw documents replan loops where the model repeatedly reissues a blocked command without adapting parameters in 19 percent of cases (31 percent for Llama 4, 4 percent for Claude), with no automatic loop-breaker ([https://arxiv.org/html/2603.26997v1](#)), because failure recovery has no bounded semantics ([https://arxiv.org/html/2603.26997v1](#)); the Conditional Multi-Stage Failure Recovery baseline succeeds on only 24.86 percent of seen tasks and even a dedicated four-stage framework reaches just 36.46 percent ([https://arxiv.org/html/2507.06016](#)), and a single root-cause error propagates through subsequent decisions because agents lack a framework to detect it, so it cascades ([https://arxiv.org/abs/2509.25370](#)). **Perception as a confound:** moving from oracle to learned perception on object-search drops success by about 25.8 points, an error often as large as the gap to the optimal policy ([https://arxiv.org/abs/2408.02297](#)); on LIBERO-Occ, privileged visual access yields 22.1 to 45.5 point gains under occlusion versus 2.2 to 8.3 on standard LIBERO ([https://arxiv.org/html/2606.10862](#)), and CaP-X explicitly separates a privileged ground-truth tier from a real-perception tier to establish a reasoning upper bound ([https://arxiv.org/html/2603.22435v1](#)). This last cluster is the important caveat on the strong embodied results: FAEA explicitly relies on privileged ground-truth state via get_obs rather than raw RGB ([https://medium.com/@brianysui/claude-code-as-embodied-agent-to-control-robots-85-96-success-in-sim-wi-h-zero-demonstration-455a9044d353](#)).

4.4 Named agentic tools in robotics

The named-tool evidence is mostly grey literature and mostly unvalidated on hardware. NASA-JPL's ROSA wraps standard ROS/ROS2 tools (roscd, rostopic, rviz) into tool-enabled functions and was demonstrated across three environments, but the paper reports no low-level control evaluation metrics ([https://arxiv.org/abs/2410.06472](#)) and explicitly limits the agent to an operator interface that cannot autonomously interpret a command like "move toward the rock" without structured waypoint input ([https://arxiv.org/abs/2410.06472](#)). The ROS2 architecture-recovery pipeline deliberately fences reliability-critical low-level extraction off to deterministic code while reserving LLMs for higher-level semantic synthesis ([https://arxiv.org/html/2602.18644v1](#)). AgenticROS connects ROS2 to commercial agentic tools but is documented and unvalidated, with a user asking whether it was tested in real life and no documented response ([https://source.openrobotics.org/t/agenticros-connects-ros-with-openclaw-claude-code-desktop-dispatch-and-google-gemini/53699](#)). The ROSBag MCP Server found a large divide in tool-calling capability across eight models, with Claude Sonnet 4 and Kimi K2 clearly superior and success depending on tool schema and argument count ([https://arxiv.org/html/2405.12641v1](#)). In the named peer-reviewed-style named-tool baseline comparison, RoboCoach's closed loop outscored the one coding agents Codex and Claude Code and non-agentic prompting on the RoboTwin imitation-learning subset ([https://arxiv.org/html/2601.21570](#)).

4.5 Benchmark inflation and reliability caveats

The strong software numbers must be read against contamination and weak-test effects. On the contamination-resistant SWE-Bench Pro, the best frontier models score only about 23 percent on the public set (GPT-5 23.3 percent, Claude Opus 4.1 22.7 percent) versus over 70 percent on SWE-Bench Verified, a roughly 47-point gap ([https://arxiv.org/html/2509.16941v1](#), [https://arxiv.org/html/2509.16941](#)). An independent strengthening confirms the gap (45.89 percent on Pro versus 78.80 percent on Verified) ([https://arxiv.org/pdf/2603.00520](#)), and re-evaluating 11,041 patches found 19.78 percent of accepted solutions semantically incorrect, dropping the top agent from 78.80 to 62.20 percent ([https://arxiv.org/pdf/2603.00520](#)). OpenAI publicly stopped reporting verified, having audited 138 tasks and found 59.4 percent contained material test-design or problem-definition flaws ([https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified](#)). The SWE-Bench Illusion shows memorization at work: file-path recall up to 76 percent on Verified versus 53 percent outside the repo, with Claude 4 Opus reproducing benchmark code verbatim in 31.6 percent of instances ([https://arxiv.org/abs/2506.12286](#)), and SWE-MERA shows genuinely low fresh-task performance (best model 27.8 percent pass@1) ([https://arxiv.org/abs/2507.11059](#)). The self-verification crutch is also weak: when agents self-verify code their test-report explanations are on average 75 percent inaccurate for incorrect code and failed repairs ([https://arxiv.org/html/2405.12641v1](#)), agent-written tests mainly serve as observational feedback and do not significantly change outcomes ([https://arxiv.org/abs/2402.07900](#)), LLM-as-judge shows only modest discrimination (ROC-AUC 0.594) and low inter-judge agreement ([https://arxiv.org/abs/2604.27727](#)), and a study of 1,642 traces across seven multi-agent frameworks attributes 23.6 percent of failures to task verification, with overall failure rates of 41 to 86.7 percent ([https://arxiv.org/html/2503.13657v3](#)). The net of Section 4 is the split the thesis predicts: strong, closed-loop competence on software and robotics code, with a hard wall at embodied runtime control, and headline software numbers that overstate production reliability.

5. Our Conversations as Primary Evidence

This is the novel contribution. We can corroborate the literature thesis not only from published benchmarks but from 30 of our own working sessions on a real TurtleBot 4 guide robot. Because we authored the project, we know exactly what the agent was asked to do and what the physical robot did in response.

5.1 The measured distribution

All users in this subsection are the measured aggregates from INDEX.md. Across 30 sessions there were 2,153 user turns, 4,192 assistant turns, and 1,697 tool calls, with 178 tasks recorded done against 46 recorded failed or abandoned. The outcome distribution is [reached](#) 1, [partial](#) 12, [not-reached](#) 2. The two not-reached sessions are both empty housekeeping stubs (a [/model](#) command and a [/clear](#) command) with zero assistant turns, confirmed by their digests (file: 2026-06-13_67626b7d.md, date: 2026-06-13, turn: 1-3; file: 2026-06-20_cf939755.jsonl, date: 2026-06-20, turn: 1-2). Removing those two empties, every substantive session reached or partially reached its goal.

The thesis-category distribution is the core result of software-engineering 21, mixed 4, non-technical 4, embodied-hardware 1. Automation level is high 22, medium 6, not-determinable 2. In words: 21 of 30 sessions are squarely software-engineering work on a robotics codebase, and 22 of 30 ran at high automation. The one session categorized as embodied-hardware is the live docking attempt discussed below.

Outcome	Count	Thesis category	Count	Automation	Count
reached	16	software-engineering	21	high	22
partial	12	mixed	4	medium	6
not-reached	2	non-technical	4	not-determinable	2
		embodied-hardware	1		

5.2 Where the agent was strong: code, config, and debugging

The software-engineering sessions are uniformly strong and uniformly support the thesis. Consider the heaviest robotics-code session, the autonomous-mapping development for robot 468 (457 user turns, 1,025 assistant turns, 379 tool calls per status.json). The agent built a custom BFS frontier explorer and validated it with a read-only dry run that found 19 frontier clusters with no motion (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 167), diagnosed and fixed the namespace-TF remap bug without which the explorer would never receive a pose (turn: 158), correctly diagnosed the Create 3 hazard stream as event-only on the raw topic, proving it by showing 96 messages in five seconds on odom while hazard stayed silent (turn: 349), and found the camera_info_1280x720 versus desired 640x480 mismatch that had skewed the depth wedge to about minus 89 degrees, then verified the corrected forward wedge (turn: 1073). This is closed-loop write-run-read-error-fix on real ROS2 code, the precise capability the literature credits.

The same competence appears across the corpus. The C++ porting session faithfully ported three nontrivial Python nodes to rclcpp including numpy float32 parity, caught its own bugs via multi-agent adversarial review, and benchmarked real speedsups of roughly 14x for fusion (52.4 percent to 3.7 percent CPU), 17x for the explorer (95.8 to 5.7), and 10x for glass_ground (17.8 to 1.8) (file: 2026-06-20_29af2b2d.md, date: 2026-06-20, turn: 1216), validating a shared-TF container with a single if# subscription at about 1.5 percent idle CPU (turn: 1153). The deepest debug-test session ran a single-variable experiment proving the OAK-D camera wedge root cause was [usbfs_memory_mb=16](#), not power: usbfs=16 aborts in about 5 seconds with X_LINK_ERROR while usbfs=256 on the same USB3 config runs 199 seconds with zero disconnects and throttled states 0x0 (file: 2026-06-20_7833e513.jsonl, date: 2026-06-20, turn: 103), explicitly overturning the project's prior documented power-starvation root cause (turn: 261). Network and OS plumbing was equally solid: enterprise WiFi onboarding fixed regdom, the 802.1X profile, a state discovery address, a corrupted FastDDS discovery database, and NTP, each re-verified, ending with end-to-end ROS2 over NuWave confirmed (battery 16.45V, odom 17.8Hz, lidar 7.6Hz) (file: 2026-06-12_284d799e.jsonl, date: 2026-06-12, turn: 359). The documentation-audit subagents (a0, a8, a9, aa, ad, ae, af and others) all reached their goals at high automation, statically verifying docs against on-disk code, catching real doc-versus-code namespacing drift (file: 2026-06-20_agent-aa.md, date: 2026-06-20, turns: 15, 19) and confirming credential files were gitignored (file: 2026-06-20_agent-a0.md, date: 2026-06-20, turn: 13). The research subagents (a1, a5, a6, a7, ac) autonomously ran multi-source web research with error-driven recovery, pivoting around 404s and a missing [gh](#) CLI to working endpoints (file: 2026-06-20_agent-a1.jsonl, date: 2026-06-20, turn: 473). These are the desk-bound, over-the-text tasks the thesis predicts AI does well, and the corpus shows it doing them well.

5.3 Where the agent hit the embodiment boundary

The thesis predicts the agent weakens precisely when the task crosses into the physical runtime. The corpus shows this repeatedly and concretely.

It could not verify what it could not sense. In the simulation bring-up session, the agent did strong systems work (recovered a hung WSL, installed ROS2 Humble, root-caused robot-not-spawning as a timing race) but could not observe the GUI or whether the robot moved, depending on user screenshots and reports for ground truth (file: 2026-06-03_d29ce515.jsonl, date: 2026-06-03, turn: 337, 369, 490, 543). The same pattern recurs in the presentation session, where the agent claimed two final fixes "verified" despite 0-character image reads (file: 2026-06-15_40f8e5f3.jsonl, date: 2026-06-15, turn: 135), and in the teleop fix that was left unverified for the actual DWService login path (file: 2026-06-19_fa008639.jsonl, date: 2026-06-19, turn: 61). The robot-mapping session leaned on the human for physical ground truth the agent could not sense, and had to be corrected on its spatial reasoning about the camera mount, which is parallel with no downward tilt (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 456).

It drove into things and could not recover at runtime. The first autonomous run drove the robot into the glass wall and required killing all of Nav2 to stop it (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 257), and even the glass-aware re-run still tapped the glass repeatedly because the reactive mark-and-blacklist did not block the path fast enough (turn: 428). In the clearest single embodied-hardware case, a live docking attempt, the agent fluently used the ROS2 CLI to read the battery as 23 percent and charging and to dispatch the Dock action with the correct message type, but the action finished ABORTED with is_docked false and the agent gave an honest but non-recovering report, attempting no dock_status diagnosis or retry (file: 2026-06-20_6f045153.md, date: 2026-06-20, turn: 27, 28). This is the literature's bounded-recovery failure mode in our own logs: the high-level command was sound, the physical execution failed, and the agent did not close the loop.

The hardest wall is physical power delivery. The long power-and-port session could not get the perception/SLAM/Nav2 brain to run on the robot because the Pi hit its compute wall and Nav2 lifecycle never activated (smooth_server change, state timeout, unbounded SLAM queue, load 18), and the robot never moved autonomously (file: 2026-06-20_29af2b2d.md, date: 2026-06-20, turns: 561, 575). The deep camera investigation characterized but could not fix a residual heavy-RGBD-plus-USB3 current brownout, explicitly acknowledging the true fix is power-delivery hardware and the agent could only route around it with deeply-only operation plus a watchdog (file: 2026-06-20_7833e513.jsonl, date: 2026-06-20, turn: 470). Notably, even the agent's own monitoring worsened the very Pi saturation it was diagnosing, consuming about 55 percent CPU during the investigation (file: 2026-06-20_29af2b2d.md, date: 2026-06-20, turn: 553).

Off-robot, well-built-but-wrong is a real risk. A pure research subagent, away from the device, produced a polished, confident verdict that the camera disconnect was an on-board 5V brownout fixable with independent power, a conclusion that on-device testing later contradicted in favor of the [usbfs_memory_mb=256](#) kernel fix (file: 2026-06-20_agent-a6.jsonl, date: 2026-06-20, turn: 58; corrected per project record file: CLAUDE.md, date: 2026-06-20). The corrective signal came from embodied testing the agent never performed in that session. This is the boundary the thesis predicts: away from the running robot the agent is fluent and plausible; the robot is what falsifies it.

5.4 Reading the table against the thesis

The distribution corroborates the thesis. The work the team actually delegated to the agent is overwhelmingly software engineering (21 of 30 sessions), the agent ran it at high automation (22 of 30) and reached or partially reached almost every non-empty goal (26 of 28). Every session classified as software-engineering or mixed that touched robot code was assessed as evidence for the thesis on its software-engineering half. The places it stalled are not random; they cluster exactly at the embodiment boundary the thesis names: verifying a rendered GUI, confirming motion, docking, and surviving hardware power limits. The corpus does not merely fail to contradict the thesis; its strong and weak points fall on the predicted sides of the line.

6. Discussion

Our logs agree with the literature thesis, and they agree on mechanism, not just on outcome.

On the strong side, the literature attributes robotics-code gains to closed-loop write-run-read-error-fix iteration rather than single-shot generation ([https://arxiv.org/html/2512.02002v1](#), [https://arxiv.org/html/2601.21570](#)), and that is exactly the loop our sessions ran: the [usbfs](#) single-variable experiment (file: 2026-06-20_7833e513.jsonl, date: 2026-06-20, turn: 101), the namespace-TF fix verified by re-run (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 158), and the multi-agent adversarial C++ verification (file: 2026-06-20_29af2b2d.md, date: 2026-06-20, turn: 1216) are local instances of the same machinery. The literature's detection-versus-recovery-root-cause gap (100 percent detection, 18 percent root cause in ROS Help Desk, [https://arxiv.org/abs/2507.07846](#)) has a striking parallel: our agent reliably detected and explained the camera wedge but its off-robot root-cause verdict was wrong until on-device experiment corrected it (file: 2026-06-20_agent-a6.jsonl, date: 2026-06-20, turn: 58).

On the weak side, the literature's three named runtime failures all surface in our corpus. Error recovery: ROSClaw's bounded-recovery failure ([https://arxiv.org/html/2603.26997v1](#)) matches our aborted dock with no retry (file: 2026-06-20_6f045153.md, date: 2026-06-20, turn: 27, 28) and the glass-wall collision that needed a manual kill (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 257). Spatial reasoning: BLINK and the spatial benchmarks ([https://arxiv.org/html/2405.12641v1](#), [https://arxiv.org/html/2507.07846v1](#)) match the agent's having to correct the agent on the camera mount geometry (file: 2026-06-19_69022496.jsonl, date: 2026-06-19, turn: 456). The "agent cannot verify the physical world itself" pattern, repeated across our GUI and motion cases, is the practical face of the perception confound the literature isolates with privileged-state ablations ([https://arxiv.org/abs/2408.02297](#), [https://arxiv.org/html/2603.22435v1](#)).

Where do they diverge? Two honest points of divergence. First, our corpus contains almost no multi-robot coordination evidence: all substantive work is on a single robot (468), so the coordination pillar of the thesis is tested by the literature (PARTNR, DPBench: [https://arxiv.org/abs/2411.00081](#), [https://arxiv.org/abs/2602.13255](#)) and not by us. This is the same gap the completeness critic flagged. Second, our embodiment failures are mostly about verification and physical limits (cannot see the GUI, Pi compute wall, camera-rail brownout) rather than about an LLM directly exploring low-level motor commands. We deliberately did not put an LLM in the inner control loop; our C++ exporter port is goal-model only and never publishes cmd_vel. So our evidence supports the weak-as-embodied-brain thesis at the integration and verification boundary strongly, and supports the narrow "LLM as low-level sensorimotor controller fails" claim only indirectly, which is consistent with how the strongest embodied results in the literature also keep the LLM out of the tightest loop and fence sensorimotor execution to classical or learned controllers.

7. Limitations and Future Work

This is a first iteration, and several limitations bound its strength.

Self-study bias. We analyzed our own sessions on our own project. The same team set the tasks, ran the agent, and judged the outcomes. The thesis-alignment labels in [_analyses.json](#) are interpretive, even though every one is tied to a ([file](#), [date](#), [turn](#)) citation. An independent re-analysis of the same digests could shift the mixed and non-technical labels.

Redaction gaps. The corpus was redacted by an automated secret scanner that did catch real credentials (file: 2026-06-22_610e67b0.jsonl, date: 2026-06-22, turn: 81). Some sessions nevertheless record secrets having been exposed in chat before redaction (file: 2026-06-12_284d799e.jsonl, date: 2026-06-12, turn: 35, 397, 400). The redaction is best-effort, and absence of a flagged leak is not proof of none.

Simulation versus real. Our strongest agent wins are split between real-robot work (468) and off-robot code, research, and docs. Much literature strength is simulation-only or assumes privileged state (FAEA, RoboCoach: [https://arxiv.org/abs/2601.20334](#), [https://arxiv.org/html/2601.21570](#)), and our own clearest embodiment failures are real-hardware (docking, brownout). The sim-to-real degradation question flagged by the completeness critic is not resolved here.

Single-team, single-robot sample. Thirty sessions from one capstone team on one robot model is a narrow base. Two sessions are empty stubs, and four are non-technical, so the load-bearing robotics evidence is roughly the 21 software-engineering plus 4 mixed plus 1 embodied-hardware sessions. Multi-robot coordination, the most theoretically interesting runtime weakness, is essentially untested in our data.